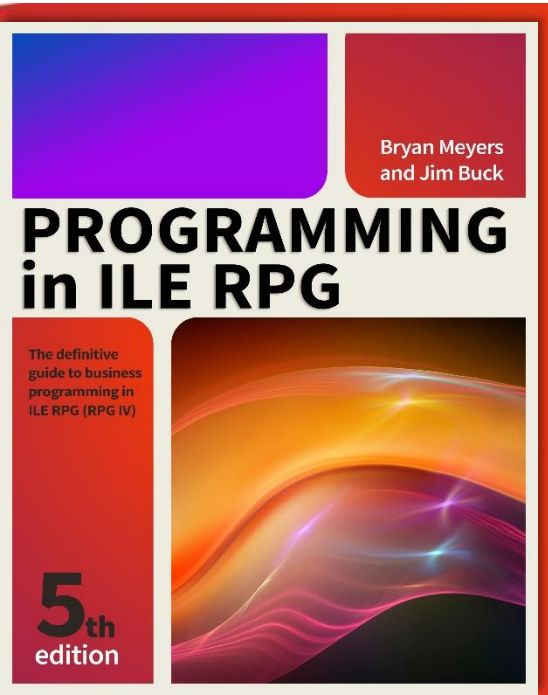
Chapter 2: Getting Started



Presentation © Copyright 2015, Enskill.com

Audio Clips

- Please listen to the audio clips found in these slides

Cycle-Main programs

- This course focuses on writing RPG code in “free format” format, and writing the logic for the program to follow
- Later in the course, we will examine “fixed format” code, also called “legacy” code.
- In fixed format, the programmer can rely solely or in part on the “RPG Cycle” for many aspects of the logic – we will examine the “RPG Cycle” in more detail in the *Fixed vs. Free* Unit of study
- HOWEVER, even when we write in free format, certain aspects of the program’s execution are *still controlled* by the “RPG Cycle.” The programs we write in the first RPG course are called “cycle main” programs for that reason

Cycle Main Programs, continued

- In some languages, such as C and Java, the programmer must explicitly name a “main” method or function. In RPG, because the RPG Cycle is controlling your program “main” does not have to be explicitly named in a cycle-main program
- To what degree the RPG Cycle controls certain aspects of the program depends on the way you write your code
    - In 100% fixed, the RPG Cycle controls many steps automatically
    - In free-form code, the RPG Cycle is still controlling many aspects of the program that in other languages would have to be explicitly coded

Cycle Main Programs, continued

- Modern RPG programmers prefer to code their own loops, if logic to test “indicators,” read statements, statements that control when output is produced, etc. instead of relying on the RPG Cycle to do those steps automatically (as was the practice in the “fixed format” era)
- HOWEVER, many the RPG Cycle is still in control, and even though some aspects of it are not used in modern coding, many aspects are still doing steps automatically, such as:
  - Opening files
  - Initializing storage areas
  - Allocating storage
  - Executing the statements in the main procedure section
  - Closing files
  - De-allocating storage and freeing up the resources that the program used when the program ends

General Free-Form RPG Syntax

- ***In code written prior to the "Technology Refresh" for IBM i 7.1***
  - Free format statements had to be coded in columns 8-80 (IBM i 7.1 OS)
  - This is why you will see existing code in applications written before that time starting the free format code in column 8
  - Column 6-7 had to be blank – otherwise compiler would assume using Fixed form
  - We are on IBM I 7.3 and therefore all that is needed is for the VERY FIRST LINE OF CODE to start in column 1 with **`**Free`**



- Statements begin with instruction to indicate purpose
  - Followed by zero or more keywords and values to further refine instruction
    - A keyword with its value is called a "parameter"
- Source is not case sensitive

```
Dcl-s Today Date(*Iso) Inz(*Sys);     // Today's date
```

General Free-Form RPG Syntax

- Semicolon (;) terminates statement
  - Single statement can span multiple lines of code
  - Only one statement may appear on any single line
- Comment begins with two slashes (//)
  - Comment may follow statement
  - Line may also consist solely of comment
- Blank lines also allowed



```
// This is a comment
// This is another comment

Dcl-s Today Date(*Iso) Inz(*Sys);     // Today's date

Dcl-s CityName Char(35) Inz('Muskegon'); // Initialized to 'Muskegon'
```

Sections of an ILE RPG Program

- Control options section
  - Provides default options for program
    - Starts with **Ctl-opt**
- Declarations section
  - Identifies and defines files, variables, and other data items
    - **F**ile declarations start with **Dcl-f**
    - "**S**tand-alone" variable declarations **Dcl-s**
    - "Named **C**onstants" declarations start with **Dcl-c**
- Main procedure section
  - Details processes, calculations, and procedures
- Subprocedure section – NOT COVERED UNTIL ADVANCED RPG
  - Includes declarations and processes for optional distinct functions (subprocedures) executed by main procedure section or by other subprocedures

Control Options (replaces older fixed-form “H” spec)



- Ctl-opt instructions control program’s behavior and specify certain compiler options
  - Default formats (e.g., date/time formats)
  - Changes to normal processing modes
  - Special compiler options (e.g., improve debugger performance)
  - Language enhancements affecting entire program



```
// Debugger facility is to ignore all “input” and “output”
// specs
Ctl-opt Option(*Nodebugio); // Use this
Ctl-opt Datfmt(*ISO) Timfmt(*HMS); // more on dates later
```

Declarations

- Declarations section defines *files* and *data* items program will use
- **Dcl-f instructions declare files, usage**
- **Dcl-c instructions declare constants, also called “named constants”**
- **Dcl-s instructions declare standalone variables (work variables)**
- Dcl-ds instruction declare data structures (used later in the course)

File Declarations (replaces older fixed-form “F” spec)

- Dcl-f instructions declare files
  - File name
  - Device
    - *Disk* – Database file
    - *Printer* – Report
  - File usage
    - *Usage(*Input)* – Program reads records from disk file
    - *Usage(*Output)* – Program writes new records to file
  - Other Keywords
    - E.g., overflow indicator to detect end of page



```
Dcl-f Customers Disk    Usage(*Input);
Dcl-f Custlist  Printer Usage(*Output) Oflind(Endofpage);
```

Standalone Variable Declarations (replaces the way this was done on the “D” fixed-form spec)

- Dcl-s instruction declares standalone variables (work fields), those that do not originate from disk file
  - Variable name
    - Begins with alphabetic character – see other rules in text
    - Can be up to 4,096 characters long – practical limit is much lower
  - Data type (and size)
    - Ind – Indicator (On/off Boolean flag) – signals true/false condition
  - Keywords
    - E.g., Inz to specify initial value

Standalone Variable Declaration Examples



```
// numeric variables
Dcl-s amount Packed(7:2); // 7 digits with 2 decimal positions
Dcl-s total  Zoned(10:2);

// character variables
Dcl-s suggestion Char(15);
Dcl-s description VarChar(60);

// indicator
Dcl-s Endofpage Ind Inz(*On);
```

Constant / Named Constant Examples

```
// without the optional CONST keyword
Dcl-c lowerLimit -50;
Dcl-c taxRate .06;
Dcl-c College 'Muskegon Community College';

// with the optional CONST keyword
Dcl-c lowerLimit CONST(-50);
Dcl-c taxRate    CONST(.06);
DCL-C College    CONST('Muskegon Community College')
```



Main Procedure Section

- Main procedure outlines processes, calculations, and procedures that program will execute
  - First part of that executes when program is called
  - Executes sequentially from beginning to end, unless operation redirects flow
- Cycle main program
  - Main procedure has no explicit designation or name
    - Follows declarations



```
Read Customers;

Dow Not %Eof(Customers);

  If Endofpage;
    Write Header;
    Endofpage = *Off;
```

RPG Operations

- Read (Read sequentially)
- Dow (Do while), Enddo
- If
- Write (Write a record)
- Eval (Evaluate expression)
- Return (Return to caller)

```
Read Customers;

Dow Not %Eof(Customers);

  If Endofpage;
    Write Header;
    Endofpage = *Off;
  Endif;

  Count += 1;
  Write Detail;
  Read Customers;
Enddo;

If Endofpage;
  Write Header;
Enddo;

Write Total;
*Inlr = *On;
Return;
```

Read (Read Sequentially)

- Read operation retrieves next sequential record from input file
  - Usage(*Input)
- Makes all field values in record available for processing
- Turns %Eof (End of file) function on when there are no more records to read

```
Read Customers;
```

Dow (Do While)

- Dow operation executes code block repetitively as long as condition is true
- Code block ends with Enddo operation

```
Dow Not %Eof(Customers);
  // …
Enddo;
```

If

- Executes code block once if condition is true
  - If condition is not true, code block is skipped
- Code block ends with Endif operation

```
If Endofpage = *On;
  // …
Endif;
```

Write (Write a Record)

- Write operation adds new record to output file, or prints report line
  - Usage(*Output)
- Writes current field values to record format

```
Write Header;
Write Detail;
Write Total;
```

Eval (Evaluate Expression)

- Eval operation assigns value to variable
  - Explicitly coding Eval is usually optional

```
Eval Endofpage = *Off;
Endofpage = *Off;

*Inlr = *On;

Count = Count + (1);
Count += 1;
```

Return (Return to Caller)

- Return operation returns control to program that called it
  - Operating system or another program
- Program execution stops
- *Inlr (Last record) signals computer should close files and free memory

```
*Inlr = *On;
Return;
```

Sample ILE RPG Program

```
// ----------------------------------------------------------------
// This program produces a customer listing report. The report data
// comes directly from input file Customers.
//
//      Date Written:  xx/xx/20xx, Bryan Meyers
// ----------------------------------------------------------------

// ------------------------------------------------ Control options
Ctl-opt Option(*Nodebugio);

// ------------------------------------------------ File declarations
Dcl-f Customers Disk    Usage(*Input);
Dcl-f Custlist  Printer Usage(*Output) Oflind(Endofpage);

// ---------------------------------- Standalone variable declarations
Dcl-s Endofpage Ind Inz(*On);
```

(Continued...)

```
// ---------------------------------------------- Main procedure
Read Customers;

Dow Not %Eof(Customers);

  If Endofpage;
    Write Header;          // Header is a format in the Custlist file
    Endofpage = *Off;
  Endif;

  Count += 1;              // Count is a variable defined in Custlist
  Write Detail;            // Detail is a format in Custlist
  Read Customers;
Enddo;

If Endofpage;
  Write Header;
Endif;

Write Total;               // Total is a format in Custlist
*Inlr = *On;
Return;
```

Legacy Code: Fixed Format

- Control Specifications
- File Description Specifications
- Definition Specifications
- Calculation Specifications

```
 * ------------------------------------------------------------------
 * This program produces a customer listing report. The report data
 * comes directly from input file Customers.
 *      Date Written:  xx/xx/20xx, Bryan Meyers
 * ------------------------------------------------------------------

 * ------------------------------------------------- Control options
H Option(*Nodebugio)

 *                                                          File declarations
```

(Continued...)

```
 * ---------------------------------------------------- File declarations
FCustomers IF   E           Disk
FCustlist  O    E           Printer Oflind(Endofpage)
 * ------------------------------------ Standalone variable declarations
D Endofpage       S                N   Inz(*On)

 * ------------------------------------------------------- Main procedure
C                   Read      Customers
C                   Dow       Not %Eof(Customers)
C                   If        Endofpage
C                   Write     Header
C                   Eval      Endofpage = *Off
C                   Endif
C                   Eval      Count += 1
C                   Write     Detail
C                   Read      Customers
C                   Enddo
C                   If        Endofpage
C                   Write     Header
C                   Enddo
C                   Write     Total
C                   Eval      *Inlr = *On
C                   Return
```